

GateIO: Learned Dead Reckoning for UAV GPS-Outage Bridging, Tested Across Held-Out Flights

Nandini Saxena

Independent research (begun during a research internship at DIAT)

nandinisaxenawork@gmail.com

Manuscript in preparation, 2026 · Code: github.com/nandini1612/gateio

Abstract

A UAV that loses GPS for a few seconds to tens of seconds must estimate its own motion from inertial data alone. Constant-velocity extrapolation drifts badly when the aircraft turns, because horizontal velocity changes with yaw rate. We present GateIO, a small causal network that predicts GPS velocity during an outage from inertial measurements and the last known GPS-aided velocity. GateIO uses one physical idea: while the aircraft flies straight (low yaw rate) the predicted velocity is held close to the last known value; the constraint is switched off during turns so the network still learns turning dynamics.

The main lesson of this work is about evaluation. A within-flight train/validation/test split, common in this area, leaks information, because validation and test windows sit next to training windows from the same flight. Under that split GateIO looks very accurate (about 7 m mean drift over a 10-second outage). Under leave-one-flight-out cross-validation, where every test flight is fully held out, the honest number is much larger: 23.4 m mean drift over 1,390 held-out outages. Even so, GateIO reduces drift 3.6 times relative to a tuned extended Kalman filter and a constant-velocity baseline (99 m and 85 m), with the largest gain on turns (9 times). We report the leakage-free result as the true measure of cross-flight performance, and treat the gap between the two protocols as a caution for the field. Cross-flight inertial bridging on this platform is improved by learning, but it is not solved.

1. Introduction

GPS-denied flight is a normal failure mode for small UAVs. Signals drop near buildings, under bridges, and in jamming. When the gap lasts several seconds and the aircraft is manoeuvring, position error grows fast.

The standard fix is a Kalman filter with a constant-velocity or constant-acceleration model. A well-tuned filter helps on straight flight, but it has no way to know that a turn is happening, so it keeps extrapolating in a straight line and the error explodes. Learned inertial odometry offers another route: networks trained on windowed IMU data can estimate motion directly and pick up patterns a fixed motion model misses. Most such work targets pedestrians and hand-held or ground platforms; aerial data is different, with higher speeds and roll into turns.

GateIO predicts GPS velocity during an outage and integrates it to a position estimate. It adds one prior tied to flight physics: when yaw rate is low, horizontal velocity is close to constant, so the model holds the last known velocity; when yaw rate is high, the prior is switched off so the drift loss teaches the turn. Our contributions are: (1) GateIO, a small (186k-parameter) causal model with a yaw-rate-gated velocity-persistence prior and a residual output head; (2) a leakage-free evaluation showing that a within-flight split overstates accuracy by a large margin; and (3) an error analysis explaining why an earlier version failed to

generalise, with a fix that recovers most of the loss on unseen straight flight. We are direct about the ceiling: under the honest protocol, mean drift is 23.4 m and 42% of outages stay under 5 m.

2. Related Work

IONet framed pedestrian inertial odometry as regressing a displacement increment from a window of IMU data. RoNIN set strong velocity-regression baselines and a benchmark. TLIO regressed 3D displacement and its uncertainty and fused it into a filter, giving a principled way to fall back on a motion model when the network is unsure. AirIO brought this to the aerial setting, predicting body-frame velocity with uncertainty; its gains come mostly from the uncertainty term. GateIO predicts a velocity increment, like IONet and TLIO, but produces a point estimate without uncertainty, which we note as a limitation. AI-IMU learns the noise covariances of an invariant EKF rather than the motion directly. GateIO uses ALiBi attention, which biases attention by distance and extrapolates to outage lengths not seen in training. We use the MARS-LVIG aerial dataset (DJI M300 RTK).

3. Data and Preprocessing

We use five flight sequences from MARS-LVIG: three from an airport site (gnss01, gnss02, gnss03) and two from an island site (island_gnss02, island_gnss03). RTK GPS gives ground-truth velocity. Raw IMU data is cut into windows of 200 samples (1.0 s at 200 Hz) with a stride giving a prediction every 0.1 s (10 Hz). Each window has 14 channels: accelerometer (3), gyroscope (3), attitude quaternion body-to-navigation (4), GPS velocity (3, zeroed during an outage), and an outage flag (1). A sequence is 300 windows (30 s). An outage is simulated by zeroing the GPS channels for 100 windows (10 s), starting at window 100. Input channels and targets are scaled with median and inter-quartile range computed on the training set only; quaternion channels pass through unscaled. Two of the five flights (gnss03, island_gnss03) carry most of the turning.

4. Method

4.1 Architecture

GateIO has four parts. A window encoder maps each 200-sample window to a 48-dimensional token via three 1D convolutions (kernels 7, 5, 3) with group normalisation and attention pooling. An outage-step encoding adds a sinusoidal encoding of the number of steps since the outage began, reset on each GPS-aided step. A temporal backbone of dilated causal convolutions (receptive field about 25 s) is followed by one layer of causal attention with ALiBi bias. A velocity head has two branches, one for GPS-aided steps and one for outage steps, the latter taking the last known velocity. The model has 186,390 parameters. A recurrent baseline, GateIO-LSTM, replaces the convolution-and-attention backbone with a two-layer LSTM.

4.2 The yaw-rate gate

During an outage, on windows where the yaw rate is below 0.10 rad/s (straight cruise), a loss term pushes the predicted velocity change toward zero, that is, hold the last known velocity. Above the threshold (turns) the gate is off and the drift loss shapes the prediction. An earlier ungated version of this prior fought the drift loss on turns and lost; gating is what lets the prior carry real weight.

4.3 Residual output

An early version of GateIO predicted absolute velocity. It worked well within a flight but failed on held-out flights: on straight flight it drifted about 30 m where holding velocity drifted about 2 m. Two causes: the model produced a roughly constant velocity error of a few m/s on the forward axis and did not fall back to the last known velocity, which was almost exact; and the prior, as written, pulled the output toward the training-set median speed rather than toward the last known speed. The fix is to predict a residual: the outage head outputs the change in velocity over the last known value, scaled by the natural size of that change. A zero residual is exactly hold-the-last-velocity, so when the input shifts on an unseen flight the model degrades to persistence instead of to a biased guess. This is the same idea as displacement-increment prediction in the learned-odometry literature. With the residual head and a corrected prior, straight-flight error on held-out flights drops back toward the persistence floor.

4.4 Training

AdamW, learning rate $8e-4$, one-cycle schedule, batch size 16, up to 200 epochs with early stopping. Turn windows are up-weighted. Loss terms: a data term on GPS-aided steps, a drift term on outage steps, the gated velocity-persistence prior, and small smoothness and cumulative-position terms.

5. Evaluation Protocol

5.1 Two splits

Within-flight split: each flight is cut 80/10/10 into train/validation/test in time order, so validation and test windows are the last parts of flights whose earlier parts are in training. Because a 30 s sequence and a 25 s receptive field are long relative to the split boundaries, and dynamics change slowly, the validation and test sets are close to the training distribution. Leave-one-flight-out cross-validation: whole flights are assigned to train, validation, and test. We run five folds; each flight is the test flight once, with three flights for training and one held-out flight for validation and model selection. No test window shares a flight with any training window.

5.2 Why the difference matters

Under the within-flight split GateIO reaches about 7 m mean drift, but lower validation drift there can mean worse held-out-flight performance: the split rewards fitting flight-specific patterns, so it corrupts not only the reported number but also model selection. Leave-one-flight-out removes both problems, at the cost of a much smaller effective dataset (five folds, one flight each for validation and test).

6. Results

6.1 Leave-one-flight-out (the honest result)

Pooled over 1,390 held-out sequences across all five flights. A sequence is labelled Turn if its mean yaw rate during the outage exceeds 0.10 rad/s, else Straight. Endpoint drift in metres; lower is better.

Group	GateIO	EKF	Const-v	GateIO <5 m
Straight (1200)	20.8	55.9	41.4	48%
Turn (190)	40.0	374.8	360.6	6%
All (1390)	23.4	99.5	85.0	42%

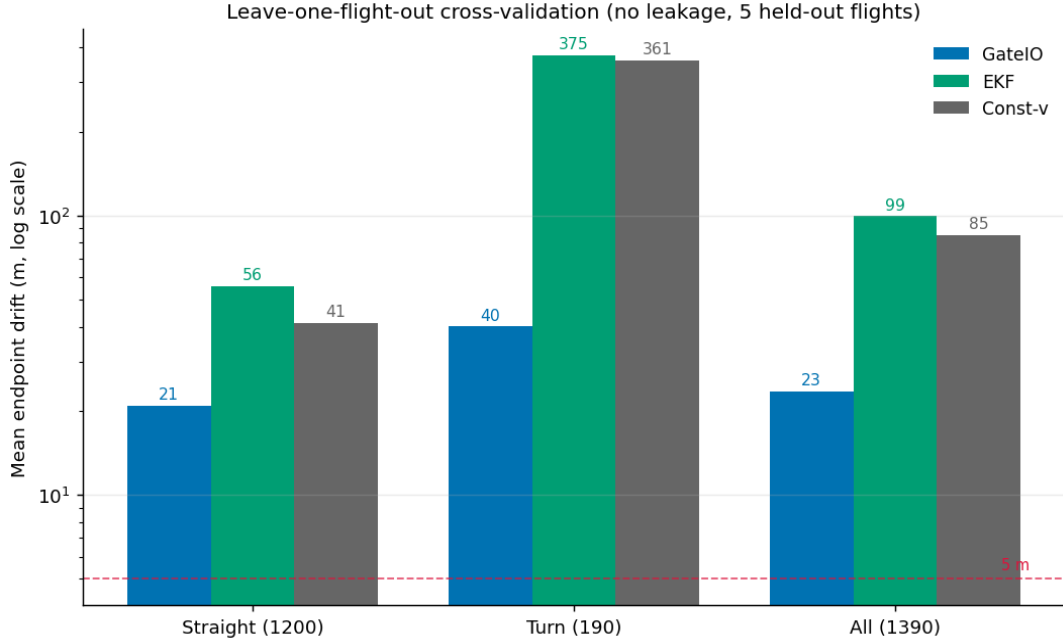


Figure 1. Leave-one-flight-out drift (log scale). GateIO is consistently below both classical baselines; the gain is largest on turns.

Per test flight (all sequences): gnss01 27.3, gnss02 13.1, gnss03 17.6, island_gnss02 23.9, island_gnss03 35.0. GateIO cuts drift 3.6 times relative to constant-velocity and 4.2 times relative to the tuned EKF; 9 times on turns and 2 times on straight flight. The constant-velocity baseline is far from perfect on straight flight here (41 m): even low-yaw sequences have real speed changes across flights, which is why a learned velocity model helps and pure persistence does not.

6.2 Within-flight (for contrast, not the headline)

Under the within-flight split the same model reaches about 7 m mean drift, with straight sequences near 1-2 m. We report this only to show how much the split inflates the number; the gap between 7 m and 23 m is the size of the leakage.

Metric (within-flight)	GateIO	LSTM	EKF	Const-v
Mean, val (m)	6.72	7.78	74.80	30.64
Mean, test (m)	34.11	42.50	58.16	62.97
% < 5 m, val	76.3%	79.7%	13.6%	74.6%
% < 5 m, test	0.0%	0.0%	11.9%	84.7%

6.3 Diagnosis

A substitution test isolates the cause. On held-out flights, if we replace the model's output with the last known velocity on gated straight windows only, straight-flight drift falls from about 30 m to under 1 m. The loss was entirely in the learned straight-flight prediction, not the architecture and not the turns. This motivated the residual head in Section 4.3.

7. Discussion

GateIO beats classical dead reckoning on unseen flights, and it beats it most where the classical methods break: turns. The learned model captures velocity changes on straight flight that a constant-velocity model misses, and it avoids the runaway turn error of a straight-line filter. The honest cross-flight number is also a warning: the within-flight split, which is easy to set up and common, made the same model look three times better than it is. Anyone comparing GPS-outage methods on this kind of data should hold out whole flights. We think the largest remaining error source is that the model has no way to say it is unsure and hold the last velocity; a learned uncertainty output, fused with a motion model, is the direction that has helped most in related aerial work.

8. Limitations

Small dataset: five flights, two sites, one platform (DJI M300); leave-one-flight-out gives five folds, each with a single validation and test flight, so per-flight estimates have low statistical power. Modest absolute accuracy: 23 m mean drift and 42% of outages under 5 m is a research result, not a fielded navigation solution. No uncertainty and no filter coupling: GateIO gives a point estimate. Weak learned baseline: we compare to constant-velocity, a tuned EKF, and our own LSTM, not to published learned-odometry methods run on this data.

9. Conclusion

GateIO is a small causal model that bridges UAV GPS outages by predicting velocity from inertial data, held to the last known value on straight flight and free to react in turns. Tested the easy way it looks very accurate; tested honestly, with whole flights held out, it reaches 23.4 m mean drift, 3.6 times better than the best classical baseline but far from solved. We report the honest number, show why the common split inflates results, and point to more data and learned uncertainty as the next steps.

References

- Brossard, M., Barrau, A., Bonnabel, S. (2020). AI-IMU dead-reckoning. *IEEE Transactions on Intelligent Vehicles* 5(4).
- Chen, C., Lu, X., Markham, A., Trigoni, N. (2018). IONet: learning to cure the curse of drift in inertial odometry. *AAAI*.
- Cioffi, G., Bauersfeld, L., Kaufmann, E., Scaramuzza, D. (2023). Learned inertial odometry for autonomous drone racing. *IEEE RA-L* 8(5).
- Herath, S., Yan, H., Furukawa, Y. (2020). RoNIN: Robust neural inertial navigation in the wild. *ICRA*.
- Li, H., Zou, Y., Chen, N., Lin, J., et al. (2024). MARS-LVIG dataset. *IJRR* 43(8).
- Liu, W., Caruso, D., Ilg, E., et al. (2020). TLIO: Tight learned inertial odometry. *IEEE RA-L* 5(4).
- Press, O., Smith, N., Lewis, M. (2022). Train short, test long: ALiBi. *ICLR*.
- Qiu, Y., Xu, C., Chen, Y., et al. (2025). AirIO: learning inertial odometry with enhanced IMU feature observability. *arXiv:2501.15659*.
- Rao, B., Kazemi, E., Ding, Y., et al. (2022). CTIN: robust contextual transformer network for inertial navigation. *AAAI*.